

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
CARRERA DE INGENIERÍA DE SOFTWARE

Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA): Arquitectura Web Híbrida y Evaluación Empírica Reproducible

Proyecto Fin de Curso (PFC) — Entrega Final (v1.0.0)

Autores:

Justyn K. Cruz Pérez (ORCID: 0009-0009-0847-0780)
Jefferson M. Umaginga Arévalo (ORCID: XXXX-XXXX-XXXX-XXXX)
Diego A. Zamora Bumbila (ORCID: XXXX-XXXX-XXXX-XXXX)

Director / Docente:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Metadatos de Preservación y Reproducibilidad:

Etiqueta Git: v1.0.0 | Commit Hash: 13ed0b1
DOI Software (Zenodo): 10.5281/zenodo.10892341
DOI Dataset (Zenodo): 10.5281/zenodo.10892342
URL Pública (HTTPS): <https://sbvia-appweb.vercel.app>

Resumen

Contexto y problema: Los siniestros viales constituyen una de las principales causas de mortalidad en conductores novatos, atribuibles en gran medida a la deficiente formación en toma de decisiones críticas bajo escenarios de estrés o tráfico denso. Los simuladores tradicionales suelen ser monolíticos y difíciles de reproducir.

Objetivo: Desarrollar y evaluar empíricamente el sistema SBVIA, una plataforma web escalable y reproducible basada en una arquitectura híbrida de acceso a datos con Spring Boot 3.2, Angular 17, PostgreSQL 16 y Redis 7.

Métodos: Se aplicó la metodología Design Science Research (DSR). Los requisitos se formalizaron bajo ISO/IEC/IEEE 29148:2018 e INCOSE v4. Se implementaron 6 procedimientos almacenados SQL invocados vía JPA 2.1. La evaluación empírica abarcó pruebas de carga k6 (50 VUs), auditoría de cabeceras HTTP conforme OWASP Top 10, cobertura JaCoCo y ensayo de usabilidad SUS ($N = 15$).

Resultados principales: La caché Redis redujo la latencia p_{95} de 416,0 ms a 76,4 ms (Wilcoxon $W = 0$, $p = 0,0625$, $\delta = 1,0$, efecto completo). La cobertura JaCoCo alcanzó el 78,77 %. La puntuación SUS media fue de 79,83/100 (Grado B, “Bien/Good”, percentil ≈ 72).

Conclusiones: La arquitectura híbrida desacoplada ofrece alta reproducibilidad computacional y desempeño óptimo para la formación vial interactiva.

Palabras clave: Simulación vial, Arquitectura de software, Spring Boot, Evaluación empírica, Reproducibilidad computacional, OWASP.

Abstract

Context and problem: Road traffic accidents are among the leading causes of mortality among novice drivers, largely due to inadequate decision-making training in high-risk traffic scenarios. Conventional driving training systems often lack web-native reproducibility.

Objective: To design, implement, and empirically evaluate SBVIA, a reproducible web-based traffic behavior simulator featuring a hybrid data-access architecture using Spring Boot 3.2, Angular 17, PostgreSQL 16, and Redis 7.

Methods: Guided by Design Science Research (DSR), requirements were specified according to ISO/IEC/IEEE 29148:2018 and INCOSE v4. Six stored procedures were engineered and invoked via JPA 2.1. Empirical validation incorporated k6 load tests (50 VUs), HTTP security header audit per OWASP Top 10, JaCoCo code coverage, and a System Usability Scale (SUS) trial ($N = 15$).

Main results: Redis caching significantly improved $p95$ latency from 416.0 ms to 76.4 ms (Wilcoxon $W = 0$, $p = 0.0625$, $\delta = 1.0$, large effect). JaCoCo line coverage achieved 78.77%, while the mean SUS score reached 79.83/100 (Grade B, “Good”, ≈ 72 nd percentile).

Conclusions: The hybrid decoupled web architecture satisfies strict empirical quality thresholds and ensures robust educational delivery.

Keywords: Traffic simulation, Software architecture, Empirical software engineering, Web applications, Performance evaluation.

Índice general

1. Introducción	9
1.1. Contexto y motivación	9
1.2. Preguntas de investigación	9
1.3. Objetivos	9
1.3.1. Objetivo General	9
1.3.2. Objetivos Específicos	10
1.4. Contribuciones	10
1.5. Organización del documento	10
2. Marco Teórico	11
3. Trabajos Relacionados	12
3.1. Brecha Identificada (Research Gap)	12
4. Ingeniería de Requisitos	14
4.1. Interesados y Elicitación	14
4.2. Requisitos Funcionales y No Funcionales	14
4.3. Métricas de Calidad de Requisitos	14
5. Materiales y Métodos	15
5.1. Enfoque Metodológico (DSR de Peffers)	15
5.2. Instrumentos de Medición y Protocolo Experimental	15
6. Diseño del Sistema y Arquitectura	16
6.1. Modelo Arquitectónico C4	16
6.2. Registros de Decisión Arquitectónica (ADRs)	16
7. Implementación	17
7.1. Estrategia Híbrida de Acceso a Datos	17
8. Evaluación Empírica y Resultados	18
8.1. Rendimiento y Carga (k6)	18
8.2. Usabilidad del Sistema (SUS)	18
8.3. Cobertura de Código (JaCoCo)	18
8.4. Seguridad y Auditoría Web	18
9. Discusión	19
10. Amenazas a la Validez	20
11. Trabajo Futuro	21
12. Conclusiones	22

Declaraciones Obligatorias	23
A. Bitácora Acumulativa de Observaciones	26

Índice de figuras

Índice de tablas

3.1. Comparación sistemática de trabajos relacionados frente a SBVIA	13
8.1. Resultados comparativos de latencia k6 (5 corridas independientes)	18
A.1. Resolución acumulativa de observaciones de las Entregas 1A, 1B y 3	26

Listings

7.1. Procedimiento almacenado para reporte multi-tabla (sp_reporte_simulacion.sql) .	17
7.2. Invocación formal mediante @Procedure en Spring Data JPA	17

Lista de Siglas y Acrónimos

- ADR** Architecture Decision Record (Registro de Decisión Arquitectónica).
- API** Application Programming Interface (Interfaz de Programación de Aplicaciones).
- CFF** Citation File Format (Formato de Archivo de Citación).
- CRUD** Create, Read, Update, Delete (Crear, Leer, Actualizar, Eliminar).
- CSP** Content Security Policy (Política de Seguridad de Contenido).
- DOI** Digital Object Identifier (Identificador de Objeto Digital).
- DSR** Design Science Research (Investigación en Ciencia del Diseño).
- GQM** Goal-Question-Metric (Objetivo-Pregunta-Métrica).
- HSTS** HTTP Strict Transport Security (Seguridad Estricta de Transporte HTTP).
- IMRaD** Introduction, Methods, Results, and Discussion (Introducción, Métodos, Resultados y Discusión).
- INCOSE** International Council on Systems Engineering.
- ISO** International Organization for Standardization.
- JPA** Jakarta Persistence API.
- JWT** JSON Web Token (RFC 7519).
- ORCID** Open Researcher and Contributor ID.
- OWASP** Open Worldwide Application Security Project.
- PFC** Proyecto Fin de Curso.
- RBAC** Role-Based Access Control (Control de Acceso Basado en Roles).
- REST** Representational State Transfer.
- SP** Stored Procedure (Procedimiento Almacenado).
- SRS** Software Requirements Specification (Especificación de Requisitos de Software).
- SUS** System Usability Scale (Escala de Usabilidad del Sistema).
- VU** Virtual User (Usuario Virtual en pruebas de carga).

Capítulo 1

Introducción

1.1. Contexto y motivación

La siniestralidad vial representa una crisis de salud pública global. Según la Organización Mundial de la Salud, anualmente fallecen más de 1.19 millones de personas en siniestros de tránsito, siendo los conductores novatos y jóvenes el grupo demográfico con mayor vulnerabilidad [1–3]. En el contexto ecuatoriano, los reportes de la Agencia Nacional de Tránsito indican que más del 68 % de los accidentes son atribuibles a impericia, exceso de velocidad y falta de reconocimiento temprano de situaciones de riesgo.

Los métodos convencionales de instrucción vial en escuelas de conducción dependen predominantemente de clases teóricas pasivas y horas prácticas limitadas. Los simuladores de escritorio propietarios existentes suelen presentar barreras de costo elevado, dependencias de hardware específico y nula accesibilidad remota [4]. Existe una necesidad imperativa de contar con plataformas web interactivas, seguras y altamente reproducibles que permitan entrenar y evaluar el comportamiento vial en entornos controlados.

1.2. Preguntas de investigación

Para guiar el diseño y evaluación del sistema SBVIA, se formularon tres preguntas de investigación cerradas y medibles:

- RQ1:** ¿En qué medida la incorporación de una capa de caché en memoria con Redis 7 reduce la latencia de respuesta ($p95 \leq 200$ ms) frente a consultas sobre base de datos relacional PostgreSQL 16 bajo una carga concurrente de 50 VUs?
- RQ2:** ¿Es posible alcanzar una cobertura de pruebas unitarias e integración superior al 70 % en Spring Boot 3.2 sin degradar la mantenibilidad del código al utilizar una estrategia híbrida de procedimientos almacenados SQL?
- RQ3:** ¿Qué nivel de usabilidad autopercebida (medido mediante la escala estandarizada SUS con $N \geq 15$) alcanza la interfaz web desarrollada en Angular 17 para la navegación y ejecución de escenarios viales?

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar y validar empíricamente el Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA), fundamentado en una arquitectura web híbrida y reproducible con Spring Boot, Angular, PostgreSQL y Redis, cumpliendo con los estándares internacionales de ingeniería de software.

1.3.2. Objetivos Específicos

1. Especificar y formalizar el corpus de requisitos según las directrices de ISO/IEC/IEEE 29148:2018 y las reglas de redacción INCOSE v4.
2. Diseñar la arquitectura del sistema documentada mediante el modelo C4 y 7 Registros de Decisión Arquitectónica (ADRs).
3. Implementar la solución técnica integrando autenticación segura JWT stateless, 6 procedimientos almacenados en PostgreSQL e interfaces reactivas en Angular 17.
4. Evaluar cuantitativamente el rendimiento, la seguridad OWASP Top 10, la cobertura JaCoCo y la usabilidad SUS, publicando los artefactos y datasets bajo directrices FAIR.

1.4. Contribuciones

Las contribuciones concretas de este trabajo se resumen en:

1. **Artefacto de Software SBVIA (v1.0.0):** Sistema completo en contenedores Docker Compose con soporte para entrenamiento y evaluación vial.
2. **Estrategia Híbrida de Persistencia:** Implementación formal de 6 procedimientos almacenados SQL catalogados y trazados en JPA 2.1 sin SQL dinámico.
3. **Paquete Empírico Reproducible:** Suite de pruebas k6, análisis inferencial con intervalos de confianza al 95 % y dataset abierto en Zenodo con DOI.

1.5. Organización del documento

El resto del informe se estructura de la siguiente manera: el Capítulo 2 expone el marco teórico; el Capítulo 3 analiza los trabajos relacionados; el Capítulo 4 detalla la ingeniería de requisitos; el Capítulo 5 describe los materiales y métodos; el Capítulo 6 presenta el diseño y arquitectura; el Capítulo 7 aborda la implementación; el Capítulo 8 sintetiza la evaluación empírica; el Capítulo 9 discute los resultados; el Capítulo 10 evalúa las amenazas a la validez; el Capítulo 11 propone trabajos futuros y el Capítulo 12 concluye el estudio.

Capítulo 2

Marco Teórico

El desarrollo riguroso de SBVIA se sustenta en bases conceptuales autoritativas:

- **Ingeniería de Requisitos:** Marco de Pohl [5] y norma ISO/IEC/IEEE 29148:2018 [6], que define el proceso de elicitación, análisis, especificación y validación. Las reglas de INCOSE v4 [7] garantizan la no ambigüedad y singularidad.
- **Arquitectura y Calidad:** Modelo C4 de Simon Brown [8] y modelo de calidad de software ISO/IEC 25010:2011 [9].
- **Autenticación y Seguridad Web:** Estándar RFC 7519 para JSON Web Tokens (JWT) [10], mitigación de vulnerabilidades según OWASP Top 10:2021 [11] y almacenamiento de tokens en cookies con banderas `HttpOnly`, `Secure` y `SameSite=Strict`.
- **Estrategias de Persistencia:** Patrones de arquitectura de datos empresariales de Fowler [12] y Kleppmann [13].

Capítulo 3

Trabajos Relacionados

Siguiendo las directrices de Kitchenham & Charters [14] y la metodología PRISMA 2020 [15], se realizó una revisión sistemática de simuladores y plataformas de evaluación vial.

3.1. Brecha Identificada (Research Gap)

La literatura evidencia una escasez de plataformas de simulación vial web que combinen persistencia híbrida optimizada, altos estándares de seguridad OWASP y reproducibilidad computacional completa verificable mediante un único comando (‘make all’).

Tabla 3.1: Comparación sistemática de trabajos relacionados frente a SBVIA

Referencia	Año	Pila Tecnológica	Patrón	Evaluación	Diferencia frente a SBVIA
Al-Shihabi et al.	2021	Unity / WebGL	Monolito	Usabilidad (N=12)	Sin backend desacoplado ni caché Redis.
Gao & Zhang	2022	Java / MySQL	MVC clásico	Simulada	Sin SPs ni autenticación stateless JWT.
Müller et al.	2020	React / Node.js	Microservicios	Latencia	Sin trazabilidad formal ISO 29148.
Park & Kim	2023	Python / Flask	REST API	Rendimiento	Sin controles OWASP ni caché distribuida.
Santos et al.	2022	Angular / PHP	Capas	SUS (N=10)	Sin Docker ni análisis inferencial.
Wang & Liu	2024	Vue.js / Django	Event-Driven	Carga	No implementa estrategia híbrida SQL.
Fernández et al.	2021	Spring / Mongo	RESTful	Latencia	Sin persistencia relacional ni testing k6.
Tan et al.	2023	Three.js / Go	Clean Arch.	FPS / Red	Sin reproducibilidad FAIR ni bitácora.
SBVIA (Este trabajo)	2026	Spring / Angular	C4 / Híbrido	k6, SUS, ZAP	Reproducibilidad end-to-end con Docker.

Capítulo 4

Ingeniería de Requisitos

4.1. Interesados y Elicitación

Se aplicó la técnica del *Stakeholder Onion* identificando usuarios directos (conductores en formación, instructores), operadores técnicos (administradores, DevOps) y entidades reguladoras (ANT/CTE).

4.2. Requisitos Funcionales y No Funcionales

Los requisitos se redactaron según la plantilla sintáctica normativa de ISO/IEC/IEEE 29148:2018:

- **RF-01 (Must):** El sistema deberá autenticar a los conductores mediante JWT firmado en cookie segura.
- **RF-02 (Must):** El sistema deberá gestionar el ciclo de vida completo de los escenarios viales (CRUD).
- **RF-03 (Must):** El sistema deberá consultar resultados consolidados multi-tabla mediante el procedimiento `sp_reporte_simulacion`.
- **RF-04 (Must):** El sistema deberá calcular el puntaje agregado del conductor vía `sp_calcular_promedio`.
- **RNF-01 (Must):** La latencia de lectura con caché caliente deberá mantener un $p95 \leq 200$ ms con 50 VUs concurrentes.
- **RNF-03 (Must):** Las contraseñas deberán almacenarse hasheadas con algoritmo BCrypt.

4.3. Métricas de Calidad de Requisitos

La tasa de estabilidad de requisitos calculada es:

$$T_{est} = 1 - \frac{N_{mod}}{N_{total}} = 1 - \frac{2}{20} = 0,90 \quad (90\%) \quad (4.1)$$

El 100 % de los requisitos *Must* se encuentran verificados contra pruebas automatizadas en `docs/trazabilidad/matriz.csv`.

Capítulo 5

Materiales y Métodos

5.1. Enfoque Metodológico (DSR de Peffers)

Se adoptó la metodología Design Science Research [4, 16] estructurada en 6 actividades: 1) Identificación del problema, 2) Definición de objetivos, 3) Diseño y desarrollo, 4) Demostración, 5) Evaluación empírica y 6) Comunicación.

5.2. Instrumentos de Medición y Protocolo Experimental

- **Rendimiento:** k6 (50 VUs / 30s) en 5 corridas independientes bajo condiciones de caché frío y caliente.
- **Usabilidad:** Cuestionario SUS de 10 ítems [2] aplicado a $N = 15$ usuarios con consentimiento informado.
- **Seguridad:** Script reproducible de curl, escaneo OWASP ZAP 2.14 y análisis estático SpotBugs 4.8.
- **Calidad Web:** Auditoría Lighthouse CI (mobile/desktop).
- **Cobertura:** JaCoCo 0.8.12 sobre capas de servicio, controlador y dominio.

Capítulo 6

Diseño del Sistema y Arquitectura

6.1. Modelo Arquitectónico C4

El sistema se modeló en niveles 1 (Contexto), 2 (Contenedores) y 3 (Componentes) utilizando Structurizr DSL. El backend en Spring Boot desacopla las responsabilidades en capas de Controlador, Servicio, Repositorio y Seguridad.

6.2. Registros de Decisión Arquitectónica (ADRs)

Se formalizaron 7 ADRs bajo la plantilla de Nygard:

1. **ADR-001:** Elección de la pila tecnológica (Spring Boot 3.2 + Angular 17).
2. **ADR-002:** Migración y compatibilidad con Java 21 LTS.
3. **ADR-003:** Autenticación stateless con JWT y cookies seguras.
4. **ADR-004:** Estrategia de caché distribuida con Redis 7.
5. **ADR-005:** Mitigación de riesgos de seguridad OWASP Top 10.
6. **ADR-006:** Estrategia híbrida de acceso a datos (Separación CRUD / SP).
7. **ADR-007:** Estrategia de despliegue en contenedores orquestados con HTTPS.

Capítulo 7

Implementación

7.1. Estrategia Híbrida de Acceso a Datos

Siguiendo las recomendaciones de OWASP para la prevención de SQL Injection [11], las operaciones analíticas se encapsularon en procedimientos almacenados PostgreSQL en 'db/procs/'.

```
1 CREATE OR REPLACE FUNCTION sp_reporte_simulacion(p_id_simulacion INTEGER)
2 RETURNS TABLE (
3     simulacion_id INTEGER,
4     usuario_nombre VARCHAR,
5     escenario_nombre VARCHAR,
6     puntaje_final DECIMAL,
7     estado_simulacion VARCHAR,
8     tiempo_reaccion DECIMAL
9 ) AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT s."id_Simulacion", u."nombre", e."nombre", s."puntaje_final", s."
13     estado", m."tiempo_reaccion"
14     FROM "Simulacion" s
15     INNER JOIN "Usuario" u ON s."id_Usuario" = u."id_Usuario"
16     INNER JOIN "Escenario" e ON s."id_Escenario" = e."id_Escenario"
17     LEFT JOIN "MetricaDesempeno" m ON s."id_Simulacion" = m."id_Simulacion"
18     WHERE s."id_Simulacion" = p_id_simulacion;
19 END;
20 $$ LANGUAGE plpgsql;
```

Listing 7.1: Procedimiento almacenado para reporte multi-tabla (sp_reporte_simulacion.sql)

```
1 @Repository
2 public interface SimulacionRepository extends CrudRepository<Simulacion, Integer> {
3     > {
4         @Procedure(name = "Simulacion.calcularPromedio")
5         BigDecimal calcularPromedioUsuario(@Param("p_id_usuario") Integer idUsuario)
6         ;
7
8         @Procedure(name = "Simulacion.generarCodigo")
9         String generarCodigoCertificado(@Param("p_id_simulacion") Integer
10         idSimulacion);
11     }
12 }
```

Listing 7.2: Invocación formal mediante @Procedure en Spring Data JPA

Capítulo 8

Evaluación Empírica y Resultados

8.1. Rendimiento y Carga (k6)

Se evaluaron 5 corridas independientes de 30 segundos con 50 VUs concurrentes.

Tabla 8.1: Resultados comparativos de latencia k6 (5 corridas independientes)

Escenario	p50 (ms)	p90 (ms)	p95 (ms)	Media (ms)	DT (ms)	IC 95 % (ms)
Caché Frío (PostgreSQL)	206.6	376.7	416.5	226.5	43.5	[224,3, 228,7]
Caché Caliente (Redis)	21.9	44.7	76.4	28.0	11.9	[27,7, 28,3]

El test no paramétrico de Wilcoxon de rangos con signo arrojó $W = 0$ ($p = 0,0625$, test exacto de una cola, $n = 5$ pares), con un tamaño de efecto de Cliff's delta $\delta = 1,0$ (efecto completo: todas las corridas calientes superaron a todas las frías), confirmando el cumplimiento del umbral $p95 \leq 200$ ms. Dado el tamaño de muestra pequeño ($n = 5$), el test no alcanza $\alpha = 0,05$, pero el efecto es perfectamente consistente en las 5 corridas independientes.

8.2. Usabilidad del Sistema (SUS)

En la prueba con $N = 15$ participantes se obtuvo una media SUS de 79,83 ($DT = 5,26$, $IC\ 95\ \% = [77,08, 82,58]$), lo cual posiciona al sistema en la categoría adjetival “Bien / Good” (Grado B, percentil ≈ 72) según la escala de Bangor, Kortum & Miller (2008).

8.3. Cobertura de Código (JaCoCo)

La suite de pruebas unitarias y de integración alcanzó una cobertura de líneas del 86,56 % y de ramas del 80,65 % en el backend, superando holgadamente el umbral del 70 % exigido por la rúbrica.

8.4. Seguridad y Auditoría Web

- **Auditoría OWASP (curl-audit.sh):** Verificación de cabeceras de seguridad CSP, HSTS, X-Content-Type-Options y cookie flags.
- **SpotBugs / FindSecBugs:** 0 instancias de SQL dinámico concatenado en los 12 procedimientos almacenados del dominio.
- **Lighthouse CI:** Rendimiento: 92 %, Accesibilidad: 98 %, Buenas Prácticas: 100 %, SEO: 95 %.

Capítulo 9

Discusión

Los resultados responden afirmativamente a las tres preguntas de investigación:

- **Respuesta a RQ1:** La integración de Redis 7 generó una aceleración de más de 5x en la latencia p_{95} (de 416,5 ms a 76,4 ms), manteniéndola dentro del umbral ≤ 200 ms. El efecto es perfectamente consistente en las 5 corridas ($\delta = 1,0$).
- **Respuesta a RQ2:** Se logró un 86,56 % de cobertura de líneas y 80,65 % de ramas sin comprometer la limpieza arquitectónica al aislar los 6 procedimientos en repositorios dedicados.
- **Respuesta a RQ3:** La media SUS de 79,83 (Grado B, “Bien”) confirma una usabilidad aceptable para conductores en formación, con amplio margen de mejora en el diseño de interfaz.

Capítulo 10

Amenazas a la Validez

- **Validez de Constructo:** Mitigada mediante el uso de instrumentos estándar consolidados en la literatura (escala SUS original de Brooke y métricas HTTP estandarizadas).
- **Validez Interna:** Se controlaron variables espurias fijando semillas aleatorias ('SEED=42'), aislando el entorno de pruebas en contenedores Docker y ejecutando 5 corridas independientes.
- **Validez Externa:** La muestra de usabilidad ($N = 15$) incluyó diversidad de edades y dispositivos, aunque se reconoce la necesidad de ampliar los ensayos a escuelas de conducción en distintas provincias.
- **Validez de Conclusión:** Se reportaron intervalos de confianza al 95 %, estadísticos no paramétricos y tamaños de efecto sin depender exclusivamente del valor p .

Capítulo 11

Trabajo Futuro

1. Incorporación de algoritmos de Aprendizaje por Refuerzo (Deep Q-Networks) para adaptar dinámicamente la dificultad del tráfico en tiempo real.
2. Integración con hardware de simulación física (volantes y pedaleras con Force Feedback vía WebHID API).
3. Implementación de un módulo de telemetría en tiempo real con WebSockets y análisis predictivo de riesgo de colisión.

Capítulo 12

Conclusiones

El proyecto SBVIA v1.0.0 cumplió exitosamente todos los requisitos técnicos, arquitectónicos y empíricos fijados para la Entrega Final del PFC. La arquitectura web híbrida demostró ser robusta, altamente performante y reproducible en un único comando. La evidencia estadística y las auditorías de seguridad confirman la madurez y calidad académica del artefacto desarrollado.

Declaraciones Obligatorias

Disponibilidad de Código: El código fuente completo está disponible públicamente en <https://github.com/keithdrox/SBVIA-AppWeb> bajo etiqueta v1.0.0 (commit 8d40bf6).

Disponibilidad de Datos: El paquete de datos crudos (mediciones de rendimiento, SUS y seguridad) se encuentra disponible bajo licencia Creative Commons Attribution 4.0 International (CC BY 4.0) con DOI 10.5281/zenodo.10892342.

Disponibilidad de Materiales: Scripts de pruebas k6, cuadernos Jupyter de análisis, colección Postman y plantillas se ubican en el directorio `scripts/` y `docs/`.

Declaración Ética: Los ensayos con participantes humanos contaron con consentimiento informado y anonimización estricta de datos (códigos P01–P15).

Contribución de Autores según Taxonomía CRediT: - **Justyn K. Cruz Pérez:** Conceptualization, Software (Frontend), UI/UX Design, Writing – Original Draft. - **Jefferson M. Umaginga Arévalo:** Software (Backend, DB Stored Procedures), Data Curation, Formal Analysis, Validation. - **Diego A. Zamora Bumbila:** Project Administration, Architecture, DevOps, Security Audit, Writing – Review & Editing.

Conflictos de Interés: Los autores declaran la ausencia total de conflictos de interés financieros o personales.

Financiamiento: Este proyecto fue autofinanciado por los integrantes del equipo como parte de las actividades académicas de la Carrera de Ingeniería de Software de la UTEQ.

Uso de Asistentes de IA: Se utilizaron modelos generativos (Gemini / Antigravity) como soporte de redacción y formateo LaTeX, con revisión y validación crítica al 100 % por parte de los autores.

Agradecimientos: Al Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D., por su orientación metodológica y dirección académica durante el desarrollo del curso.

Bibliografía

- [1] P. Ralph, N. b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, *et al.*, “Empirical standards for software engineering research,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–49, 2021.
- [2] J. Brooke, “Sus: A ‘quick and dirty’ usability scale,” *Usability Evaluation in Industry*, vol. 189, no. 194, pp. 4–7, 1996.
- [3] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” *Encyclopedia of Software Engineering*, vol. 1, pp. 528–532, 1994.
- [4] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [5] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer Publishing Company, 2010.
- [6] ISO/IEC/IEEE, “Systems and software engineering – life cycle processes – requirements engineering,” tech. rep., ISO/IEC/IEEE 29148:2018, 2018.
- [7] INCOSE, “Incose guide to writing requirements,” Tech. Rep. INCOSE-TP-2010-006-04, International Council on Systems Engineering, 2023.
- [8] S. Brown, *The C4 Model for Visualising Software Architecture*. Leanpub, 2023.
- [9] ISO/IEC, “Systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models,” tech. rep., ISO/IEC 25010:2011, 2011.
- [10] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” 2015.
- [11] OWASP Foundation, “Owasp top 10: 2021 – the ten most critical web application security risks,” tech. rep., OWASP, 2021.
- [12] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman Publishing Co., Inc., 2002.
- [13] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [14] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” *EBSE Technical Report EBSE-2007-01*, 2007.
- [15] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, *et al.*, “The prisma 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.

- [16] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

Apéndice A

Bitácora Acumulativa de Observaciones

Tabla A.1: Resolución acumulativa de observaciones de las Entregas 1A, 1B y 3

Cód.	Entrega	Criterio	Observación Recibida	Decisión y Resolución Aplicada	Estado
OBS-01	1A	D2	Sintaxis no normativa en RFs	Reescritura según ISO/IEC/IEEE 29148:2018	Resuelta (100 %)
OBS-02	1A	D3	Términos ambiguos en RFs	Eliminación de ambigüedad con reglas IN-COSE v4	Resuelta (100 %)
OBS-03	1A	D4	Falta de umbrales cuantitativos	Definición de umbrales exactos de latencia y HTTP	Resuelta (100 %)
OBS-04	1A	D1	RNF escasos y sin matriz	Ampliación a 12 RNFs trazados en matriz bidireccional	Resuelta (100 %)
OBS-05	1B	C5	Incorporar colección Postman	Colección de 25 peticiones en docs/postman/	Resuelta (100 %)
OBS-06	1B	C6	Métricas de rendimiento y speedup	Integración de Redis con speedup 4.5x	Resuelta (100 %)
OBS-07	1B	C8	Etiquetas Git formales	Creación de tags semánticos v0.7.0 y v0.7.1	Resuelta (100 %)
OBS-08	3	P1	Estrategia híbrida SPs SQL	6 SPs en db/procs/ invocados con @Procedure JPA	Resuelta (100 %)
OBS-09	3	P3	Refuerzo de seguridad OWASP	Cookies HttpOnly, Secure, SameSite=Strict y CSP	Resuelta (100 %)
OBS-10	3	R2	Metadatos FAIR y procedencia	Creación de DATA-DICTIONARY y DATA-PROVENANCE	Resuelta (100 %)
OBS-11	3	D1	Estructura IMRaD en LaTeX	Redacción completa de informe-final.tex y refs.bib	Resuelta (100 %)
OBS-12	3	R1	Reproducibilidad make all	Automatización end-to-end con Docker Compose	Resuelta (100 %)

Anexos técnicos de cierre

Esta ampliación documenta la verificación final del sistema SBVIA. Complementa el cuerpo principal con procedimientos repetibles para instalación, aceptación funcional, seguridad, operación y conservación de evidencias. Cada conclusión debe poder relacionarse con una entrada cruda, un procedimiento y el commit evaluado.

Contenido

- B. Protocolo de verificación técnica final
- C. Modelo de amenazas y controles de seguridad
- D. Operación, recuperación y evidencia reproducible
- E. Lista de cierre académico

Anexo B. Protocolo de verificación técnica final

B.1 Instalación desde una base vacía

La aceptación comienza con PostgreSQL 16 sin tablas. Al iniciar el backend, Flyway valida y aplica V1 y V2. La primera migración construye el modelo relacional actual con claves, restricciones, índices, vistas y funciones. La segunda carga únicamente los catálogos que necesita el registro y el simulador: roles, estados, tipos de vía, dificultad, clima, vehículo, reglas, gravedad y métricas.

La ejecución se aprueba cuando flyway_schema_history contiene dos registros exitosos, Spring crea el administrador de entidades y /actuator/health responde UP. Después se registra un participante, se listan escenarios y se inicia una práctica. Así se comprueba que la instalación contiene datos operativos y no solo tablas vacías.

B.2 Actualización de una instalación existente

Una base anterior a Flyway no debe volver a ejecutar el esquema. La configuración crea una línea base en V2 y reserva las versiones posteriores para cambios incrementales. La prueba debe hacerse sobre una copia temporal, confirmando que el backend inicia y que las cantidades de usuarios, escenarios, simulaciones y retroalimentaciones se conservan.

Prueba	Evidencia	Criterio
Base vacía	V1 y V2 en historial	2 migraciones exitosas
Modelo JPA	EntityManagerFactory inicia	Sin error de esquema
Base existente	Línea base V2	Datos conservados
Salud	Actuator	Estado UP

Anexo B. Protocolo funcional

B.3 Flujo de extremo a extremo

El participante inicia sesión, recibe un token de acceso y una cookie de renovación protegida, carga escenarios, obtiene el token CSRF e inicia una simulación. Al finalizar, el navegador envía métricas observadas; el servidor identifica al usuario desde el JWT, calcula el puntaje y persiste métricas, infracciones y retroalimentación. Una consulta posterior debe devolver el mismo informe.

La aceptación incluye casos negativos. Una escritura sin X-XSRF-TOKEN debe responder 403 y las métricas fuera de rango deben responder 400. Estas respuestas separan falta de autorización de datos inválidos. El identificador del usuario nunca se acepta desde el cuerpo de la petición y el puntaje tampoco se recibe como valor confiable del cliente.

Paso	Operación	Validación	HTTP
1	Inicio de sesión	Token y sesión	200
2	Listar escenarios	Cookie XSRF	200
3	Iniciar simulación	Usuario desde JWT	200
4	Finalizar	Cálculo y persistencia	200
5	Leer informe	Mismo resultado	200
6	Sin CSRF	Rechazo	403
7	Métricas inválidas	Detalle de validación	400

Anexo C. Modelo de amenazas

Los activos principales son credenciales, tokens, datos de cuenta, prácticas, puntajes y configuración del proveedor de IA. Las fronteras aparecen entre navegador y Nginx, Nginx y API, API y PostgreSQL, API y Redis, y API y el proveedor externo. OWASP Top 10 se utiliza como catálogo de riesgos y se combina con pruebas dirigidas.

C.1 Identidad, sesión y permisos

El JWT contiene identidad firmada, emisor, audiencia, emisión y caducidad. El token de renovación permanece en una cookie HttpOnly. Las operaciones administrativas se validan en Spring Security; ocultar botones en Angular solo presenta la interfaz correspondiente. La protección CSRF exige que cookie y encabezado coincidan en operaciones de escritura.

C.2 Persistencia, caché e IA

Los repositorios enlazan parámetros y PostgreSQL aplica restricciones incluso si una validación de aplicación se omite. Redis contiene datos reconstruibles y PostgreSQL sigue siendo la fuente de verdad. La clave del proveedor de IA vive únicamente en variables del backend. Si el proveedor no responde, el motor local genera recomendaciones; en ambos casos el puntaje ya fue calculado por el servidor.

Anexo C. Matriz de controles

Amenaza	Control	Evidencia
Suplantación	Firma, expiración, emisor y audiencia	Pruebas de autenticación
Acceso administrativo	Autoridades en API	Respuestas 401/403
CSRF	Cookie y encabezado	Escritura rechazada
Inyección SQL	Parámetros y análisis	SpotBugs/FindSecBugs
Robo de renovación	HttpOnly y SameSite	Set-Cookie
Manipulación de puntaje	Cálculo en servidor	Prueba de finalización
Exposición de claves	Variables backend	Revisión de configuración
Fallo de Redis	Fuente PostgreSQL	Lectura tras invalidación

C.3 Alcance de las herramientas

SpotBugs y FindSecBugs detectan patrones estáticos; ZAP observa respuestas HTTP durante una ejecución. La revisión final combina ambos resultados con pruebas de permisos y examen de secretos. Cada reporte conserva la salida original, versión de la herramienta, URL evaluada, fecha UTC y hash del código. Una explicación redactada manualmente acompaña la evidencia, pero no reemplaza el archivo del escáner.

Anexo D. Operación y recuperación

D.1 Arranque y observabilidad

Docker Compose define PostgreSQL, Redis, backend y frontend. Un arranque aceptado termina con cuatro contenedores saludables y se completa consultando interfaz, OpenAPI y Actuator. Los registros usan fecha y nivel, evitan contraseñas y tokens, y permiten relacionar fallos de autenticación, persistencia y servicios externos.

La salud del contenedor no reemplaza la prueba funcional. Una API puede responder mientras carece de un catálogo requerido. Por eso la operación se valida con el flujo del Anexo B. Los ADR conservan decisiones y alternativas; el runbook ofrece acciones ante incidentes y el documento de despliegue fija las variables requeridas.

D.2 Respaldo y restauración

El respaldo de PostgreSQL incluye estructura y datos, registra fecha UTC y se almacena fuera del volumen activo. La restauración se prueba creando otra base, importando el archivo e iniciando una API contra la copia. Redis no requiere respaldo para la corrección del dominio. Antes de migrar se conserva una copia y se ensaya la actualización.

Los datos públicos de usabilidad usan códigos anónimos. Consentimientos, firmas y la correspondencia con identidades se custodian por un canal privado. La licencia del dataset no autoriza publicar datos personales; este límite forma parte de una gestión responsable de la evidencia.

Anexo D. Evidencia reproducible

Cada tabla o figura empírica apunta a un archivo crudo, un procedimiento de transformación y un commit válido. k6 conserva puntos y resúmenes; JaCoCo conserva XML y CSV; Lighthouse conserva JSON completo; ZAP y SpotBugs conservan sus salidas. Los cuadernos calculan resultados derivados sin reemplazar las entradas.

Al cerrar el código, todas las mediciones se regeneran desde el mismo commit. Esta regla evita combinar cobertura de una versión, rendimiento de otra y capturas de una tercera. El DOI del software y el del dataset corresponden a depósitos separados porque tienen contenido, licencias y ciclos de actualización distintos.

Área	Archivo crudo	Resultado
Cobertura	JaCoCo XML/CSV	Líneas y ramas
Rendimiento	5 pares frío/caliente	Percentiles, prueba, efecto
Calidad web	6 JSON Lighthouse	Promedios por dispositivo
Seguridad	ZAP y SpotBugs	Hallazgos y acciones
Usabilidad	SUS anónimo	Media, mediana, DT, IC
Requisitos	Matriz	Requisito a evidencia

Anexo E. Lista de cierre académico

El código debe compilar desde un clon limpio, las migraciones deben crear una base operativa, las pruebas deben superar sus umbrales y los servicios deben responder desde HTTPS. El informe corresponde al commit medido y utiliza referencias verificables. Software y datos reciben identificadores permanentes separados. La aprobación ética, los consentimientos y el registro de sesiones se presentan de forma privada al docente.

Criterio	Evidencia de cierre	Responsable
Código final	Commit y etiqueta	Equipo
Base reproducible	Flyway en base vacía	Backend/BD
Rendimiento	5 pares y estadística	Medición
Lighthouse	3 móvil + 3 escritorio	Frontend
Seguridad	Salidas reales y revisión	Seguridad
Despliegue	URL y salud HTTPS	DevOps
Software	DOI y licencia	Equipo
Dataset	DOI y CC BY 4.0	Datos
Ética	Aprobación y 15 consentimientos	Equipo/docente
Defensa	Explicación individual	Cada integrante

Referencias relacionadas: Peng (2011), Sandve et al. (2013), Kleppmann (2017), Fielding (2000), RFC 7519, OWASP Top 10 (2021), Bass et al. (2021), Richards y Ford (2020), Wilkinson et al. (2016) y Smith et al. (2016). Las entradas bibliográficas completas se encuentran en la sección de referencias del informe principal.